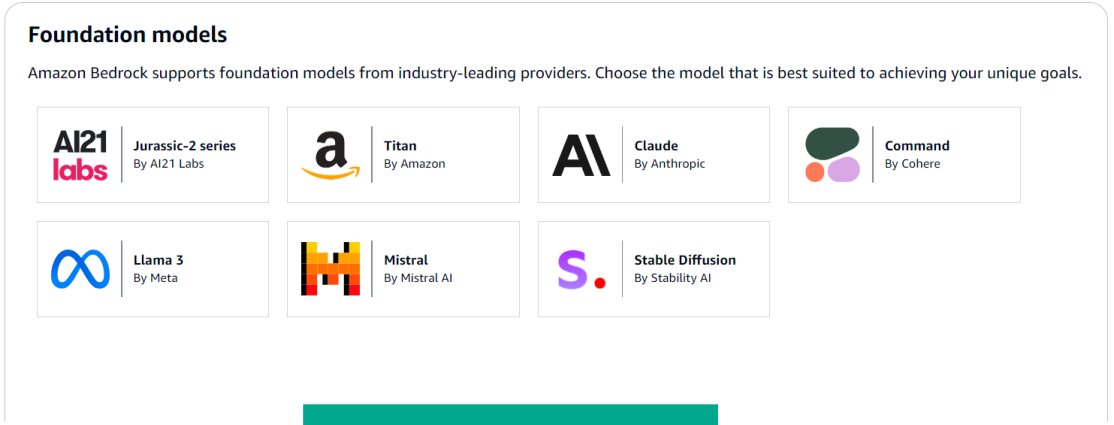
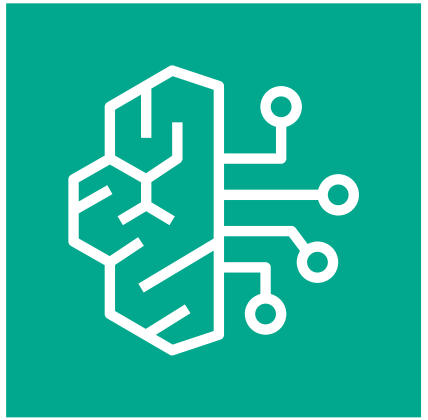
Amazon Bedrock

Using Foundation Models in AWS

Amazon Bedrock

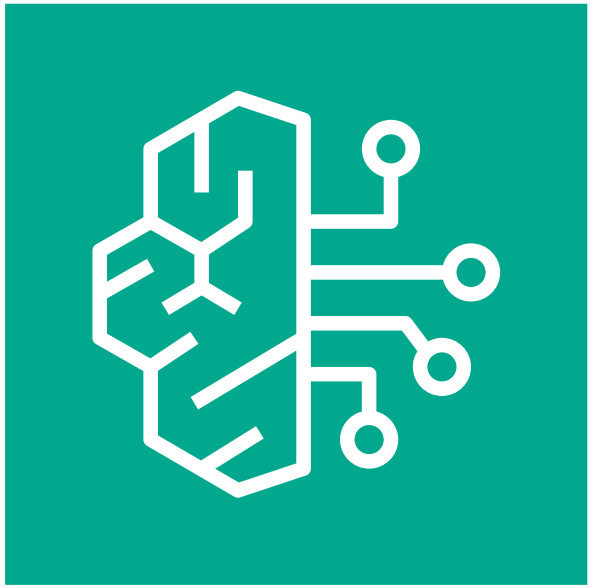
- An API for generative AI Foundation Models
  - Invoke chat, text, or image models
  - Pre-built, your own fine-tuned models, or your own models
  - Third-party models bill you through AWS via their own pricing
  - Support for RAG (Retrieval-Augmented Generation… we'll get there)
  - Support for LLM agents
- Serverless
- Can integrate with SageMaker Canvas





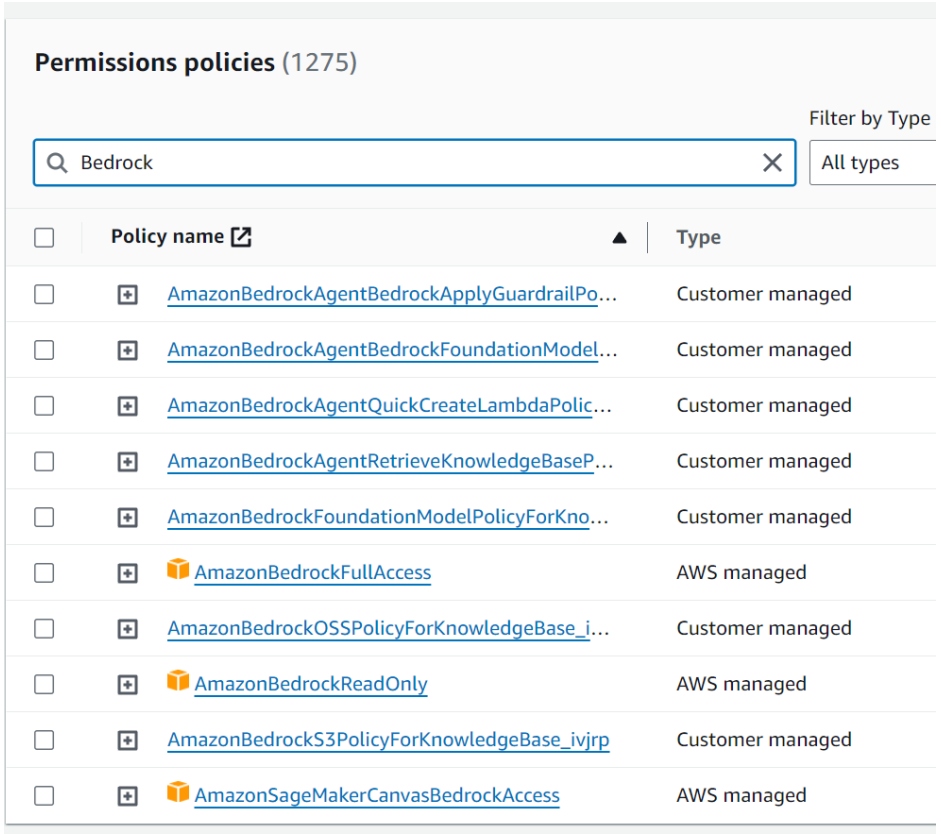
The Bedrock API Endpoints

- **bedrock**: Manage, deploy, train models
- **bedrock-runtime**: Perform inference (execute prompts, generate embeddings) against these models
  - Converse, ConverseStream, InvokeModel, InvokeModelWithResponseStream
- **bedrock-agent**: Manage, deploy, train LLM agents and knowledge bases
- **bedrock-agent-runtime**: Perform inference against agents and knowledge bases
  - InvokeAgent, Retrieve, RetrieveAndGenerate



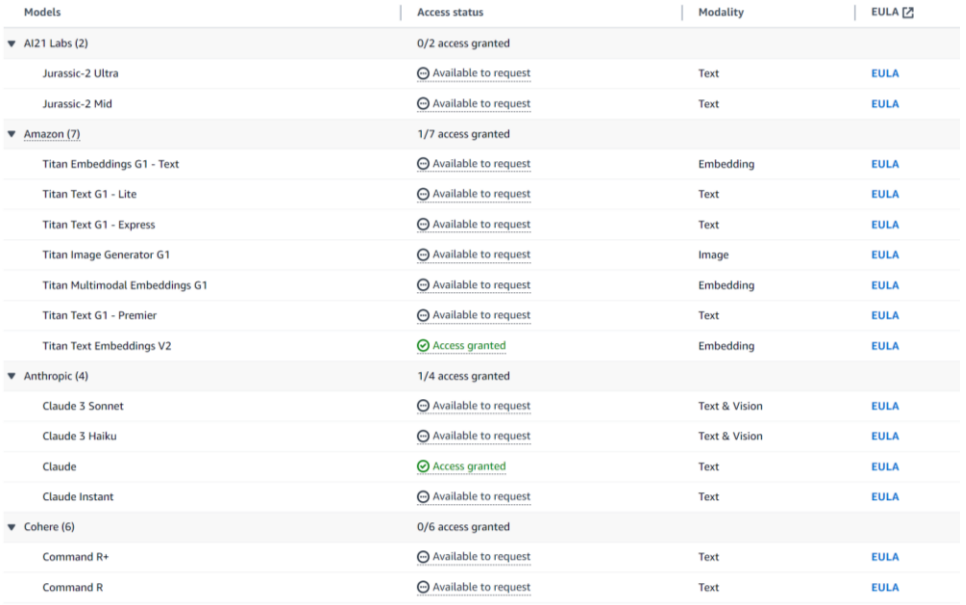
Bedrock IAM permissions

- Must use with an IAM user (not root)
- User must have relevant Bedrock permissions
  - AmazonBedrockFullAccess
  - AmazonBedrockReadOnly



Amazon Bedrock: Model Access

- Before using any base model in Bedrock, you must first request access
- Amazon (Titan) models will approve immediately
- Third-party models may require you to submit additional information
  - You will be billed the third party's rates through AWS
  - It only takes a few minutes for approval
- Be sure to check pricing
  - aws.amazon.com/bedrock/pricing



Models	Access status	Modality	EULA
AI21 Labs (2)	0/2 access granted		
Jurassic-2 Ultra	Available to request	Text	EULA
Jurassic-2 Mid	Available to request	Text	EULA
Amazon (7)	1/7 access granted		
Titan Embeddings G1 - Text	Available to request	Embedding	EULA
Titan Text G1 - Lite	Available to request	Text	EULA
Titan Text G1 - Express	Available to request	Text	EULA
Titan Image Generator G1	Available to request	Image	EULA
Titan Multimodal Embeddings G1	Available to request	Embedding	EULA
Titan Text G1 - Premier	Available to request	Text	EULA
Titan Text Embeddings V2	Access granted	Embedding	EULA
Anthropic (4)	1/4 access granted		
Claude 3 Sonnet	Available to request	Text & Vision	EULA
Claude 3 Haiku	Available to request	Text & Vision	EULA
Claude	Access granted	Text	EULA
Claude Instant	Available to request	Text	EULA
Cohere (6)	0/6 access granted		
Command R+	Available to request	Text	EULA
Command R	Available to request	Text	EULA

Let’s Play

- Bedrock provides “playground” environments
  - Chat
  - Text
  - Images
- Must have model access first
- Also useful for evaluating your own custom or imported models
- Let’s go hands-on and get a feel for it.

Fine-tuning

- Adapt an existing large language model to your specific use case!
- Additional training using your own data – potentially lots of it
  - Eliminates need to build up a big conversation to get the results you want (“prompt engineering” / “prompt design”)
  - Saves on tokens in the long run
- Your fine-tuned model can be used like any other
- You can fine-tune a fine-tuned model, making it “smarter” over time
- Applications:
  - Chatbot with a certain personality or style, or with a certain objective (i.e., customer support, writing ads)
  - Training with data more recent than what the LLM had
  - Training with proprietary data (i.e., your past emails or messages, customer support transcripts)
  - Specific applications (classification, evaluating truth)

Fine-tuning in Bedrock: “Custom Models”

- Titan, Cohere, and Meta models may be “fine-tuned”
- Text models: provide labeled training pairs of prompts and completions
  - Can be questions and answers
  - Upload training data into S3
- Image models: provide pairs of image S3 paths to image descriptions (prompts)
  - Used for text-to-image or image-to-embedding models
- Use a VPC and PrivateLink for sensitive training data
- This can get expensive
- Your resulting “custom model” may then be used like any other.

```
{"prompt": "What is the meaning of life?", "completion": "The meaning of life is 42."}
{"prompt": "Who was the best Dr. Who?", "completion": "Matt Smith in Series 5 was the best, and anyone who says otherwise is wrong."}
{"prompt": "Is Dr. Who better than Star Trek?", "completion": "Blasphemer! Star Trek changed the world like no other science fiction has."}
```

“Continued Pre-Training”

- Like fine-tuning, but with unlabeled data
- Just feed it text to familiarize the model with
  - Your own business documents
  - Whatever
- Basically including extra data into the model itself
  - So you don’t need to include it in the prompts

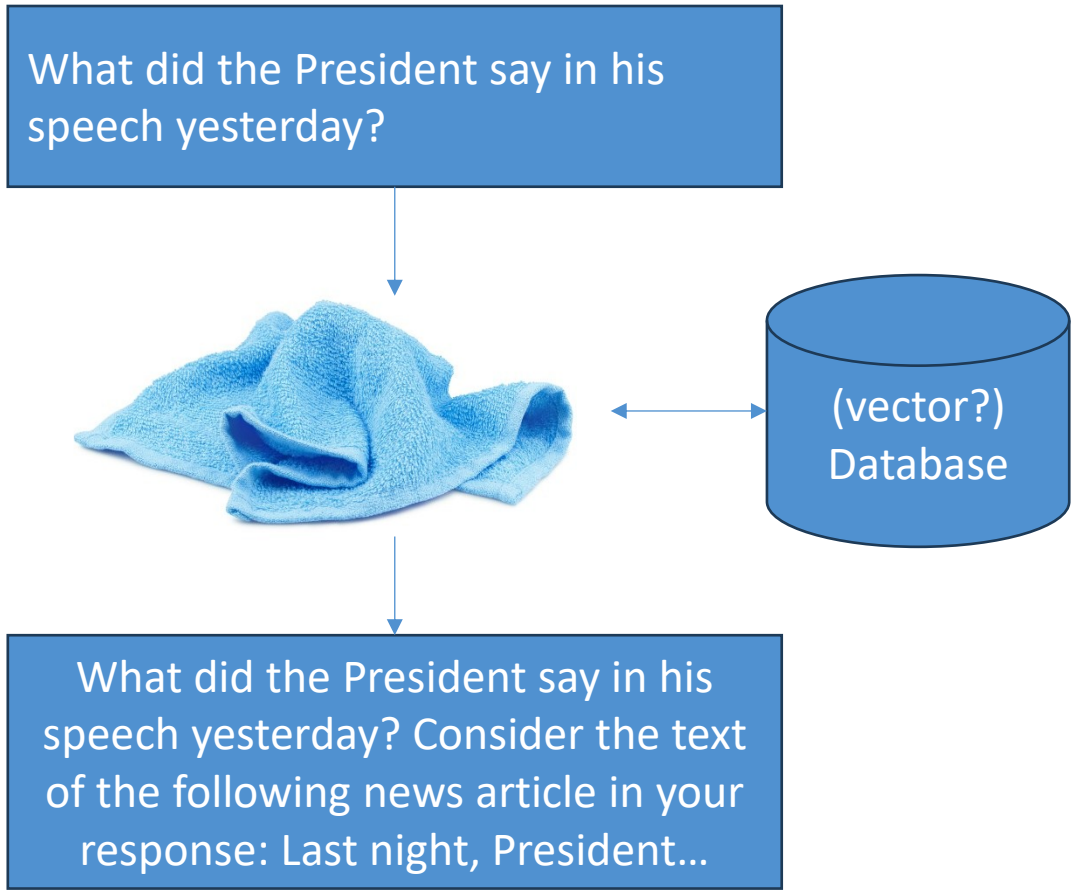
```
{"input": "Spring has sprung. "
{"input": "The grass has riz."}
{"input": "I wonder where the flowers is."}
```

Retrieval Augmented Generation (RAG)

Bedrock Knowledge Bases

Retrieval Augmented Generation (RAG)

- Like an open-book exam for LLM's
- You query some external database for the answers instead of relying on the LLM
- Then, work those answers into the prompt for the LLM to work with
  - Or, use tools and functions to incorporate the search into the LLM in a slightly more principled way

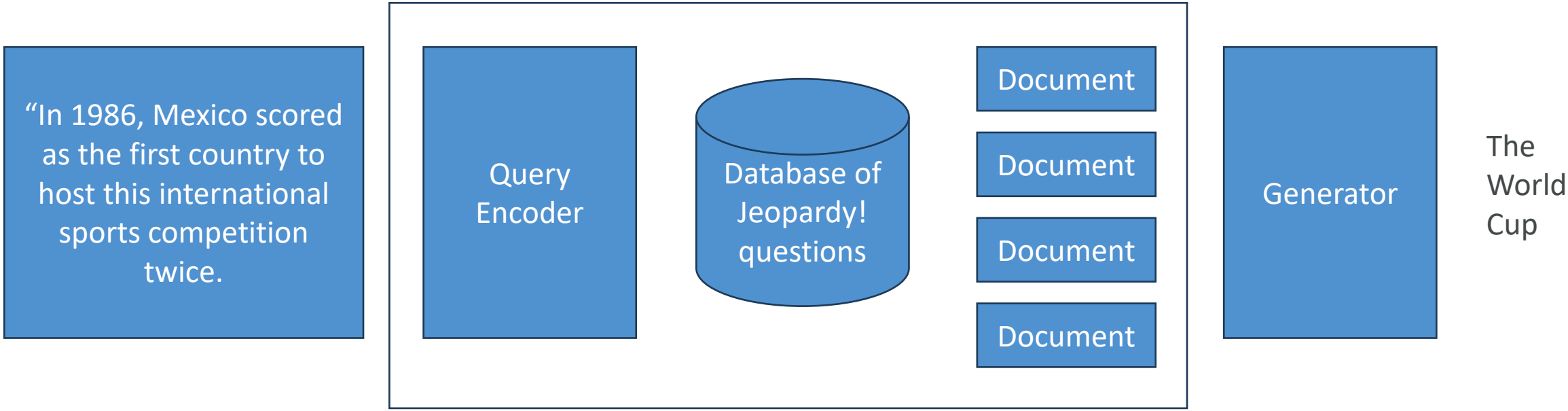


RAG: Pros and Cons

- Faster & cheaper way to incorporate new or proprietary information into “GenAI” vs. fine-tuning
- Updating info is just a matter of updating a database
- Can leverage “semantic search” via vector stores
- Can prevent “hallucinations” when you ask the model about something it wasn’t trained on
- If your boss wants “AI search”, this is an easy way to deliver it.
- Technically you aren’t “training” a model with this data

- You have made the world’s most overcomplicated search engine
- Very sensitive to the prompt templates you use to incorporate your data
- Non-deterministic
- It can still hallucinate
- Very sensitive to the relevancy of the information you retrieve

RAG: Example Approach (winning at Jeopardy!)



Basically, we are handing the generator potential answers from an external database.

Choosing a Database (Knowledge Base Data Store) for RAG

- You could just use whatever database is appropriate for the type of data you are retrieving
  - Graph database (i.e., Neo4j) for retrieving product recommendations or relationships between items
  - Opensearch or something for traditional text search (TF/IDF)
  - But almost every example you find of RAG uses a Vector database
  - Note Elasticsearch / Opensearch can function as a vector DB

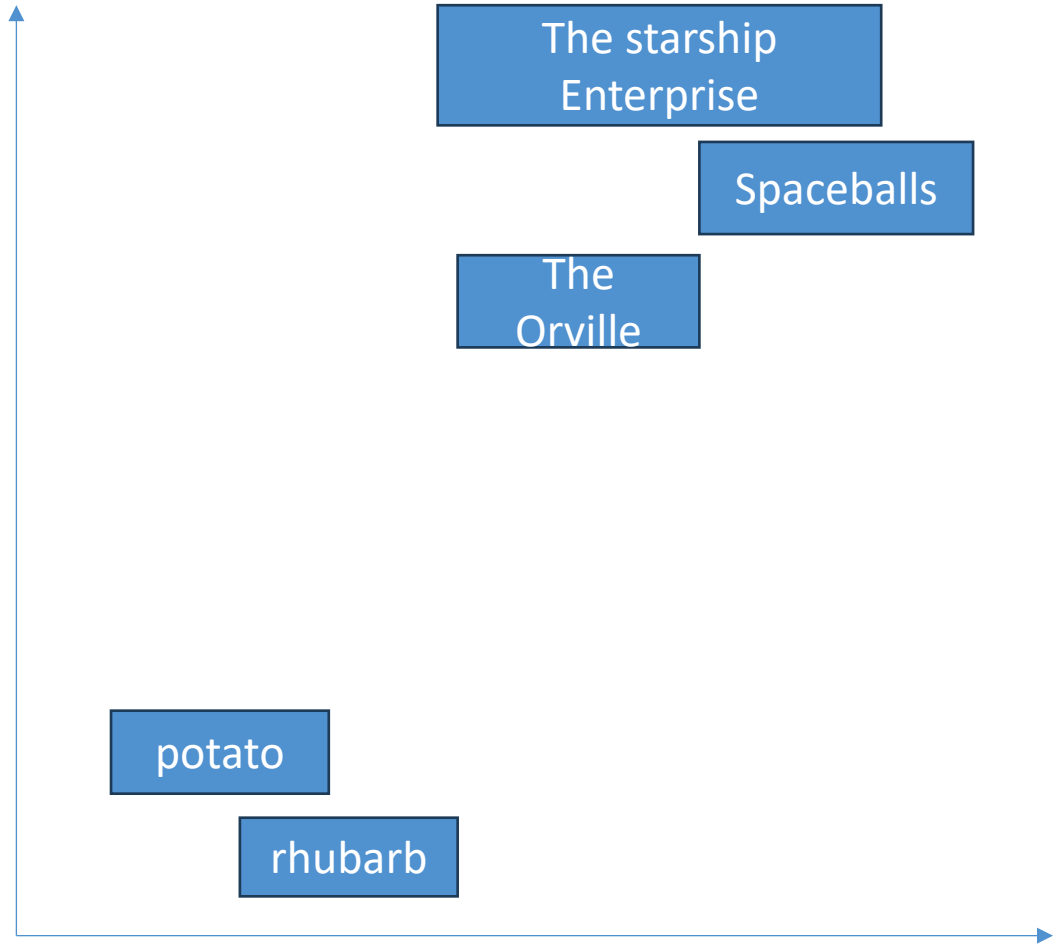
```
Q: 'Which pink items are suitable
for children?'
{
  "color": "pink",
  "age_group": "children"
}

Q: 'Help me find gardening gear
that is waterproof'
{
  "category": "gardening gear",
  "characteristic": "waterproof"
}

Q: 'I'm looking for a bench with
dimensions 100x50 for my living
room'
{
  "measurement": "100x50",
  "category": "home decoration"
}
```

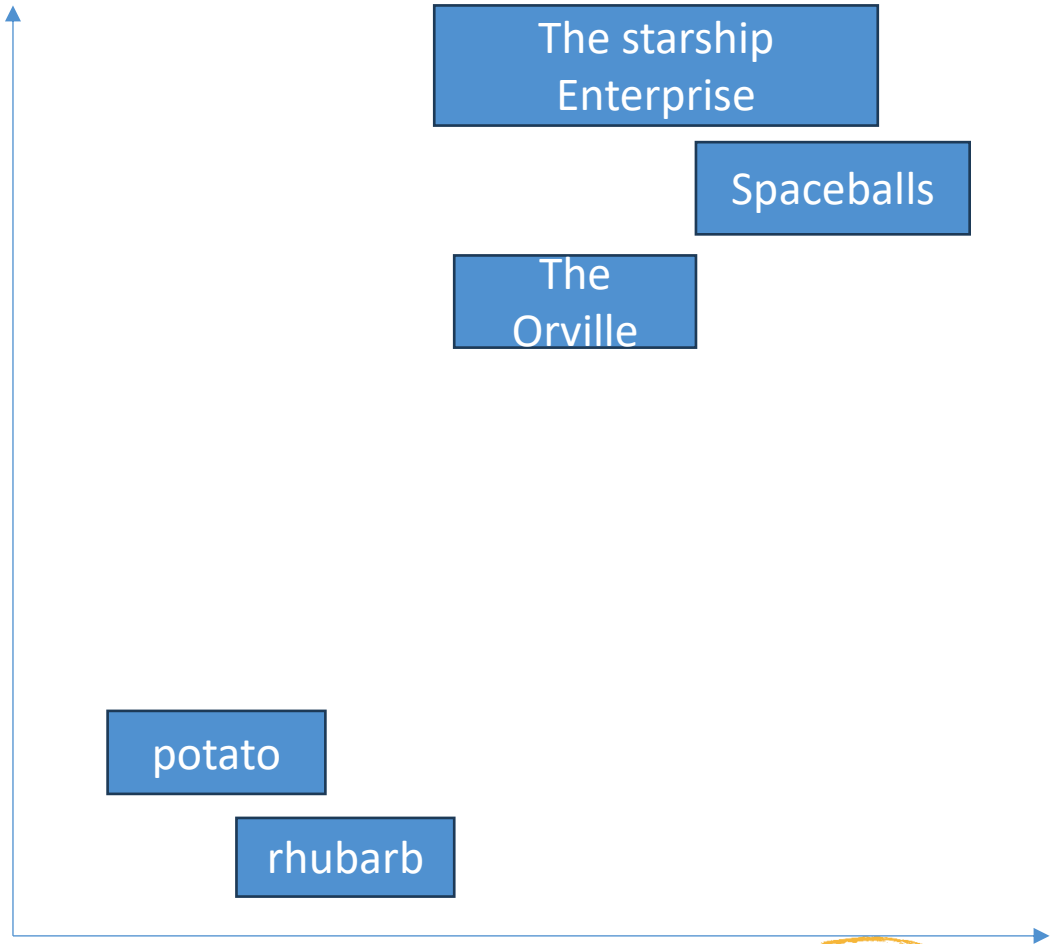
Review: embeddings

- An **embedding** is just a big vector associated with your data
- Think of it as a point in multi-dimensional space (typically 100's or thousands of dimensions)
- Embeddings are computed such that items that are similar to each other are close to each other in that space
- We can use embedding base models (like Titan) to compute them en masse



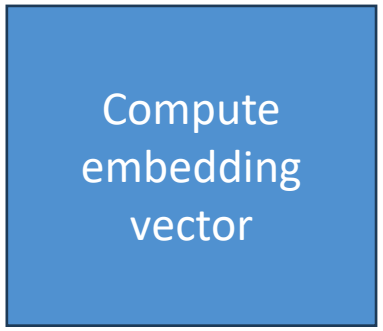
Embeddings are vectors, so store them in…

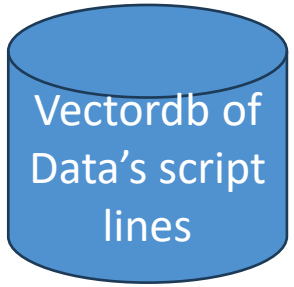
- …a vector database!
- It just stores your data alongside their computed embedding vectors
- Leverages the embeddings you might already have for ML
- Retrieval looks like this:
  - Compute an embedding vector for the thing you want to search for
  - Query the vector database for the top items close to that vector
  - You get back the top-N most similar things (K-Nearest Neighbor)
  - “Vector search”
- Examples of vector databases
  - Coercing existing databases to do vector search
    - Opensearch / Elasticsearch, SQL, Neptune, Redis, MongoDB, Cassandra
  - Purpose-built vector DB’s
    - Pinecone, Weaviate (commercial)
    - Chroma, Marqo, Vespa, Qdrant, LanceDB, Milvus, vectordb (open source)

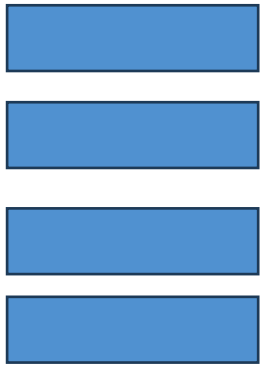


RAG Example with a Vector Database: Making Data from Star Trek, by Cheating.

Data, tell me about your daughter Lal.







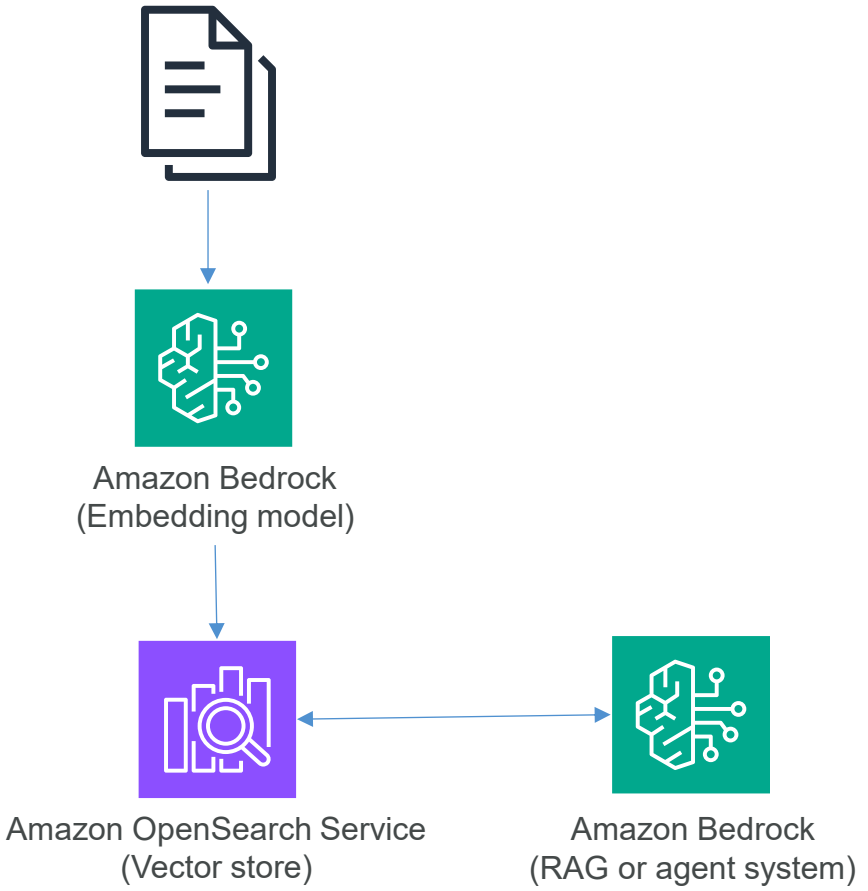
Similar lines

You are Commander Data from Star Trek. How might Data respond to the question "Data, tell me about your daughter Lal", taking into account the following related lines from Data: …

Prompt + query + relevant data

RAG in Bedrock: Knowledge Bases

- You can upload your own documents or structured data (via S3, maybe with a JSON schema) into Bedrock “Knowledge Bases”
  - Other sources: web crawler, Confluence, Salesforce, SharePoint
- Must use an embedding model
  - For which you must have obtained model access
  - Currently Cohere or Amazon Titan
  - You can control the vector dimension
- And a vector store of some sort
  - For development, a serverless OpenSearch instance may be used by default
  - MemoryDB now has vector capabilties
  - Or Aurora, MongoDB Atlas, Pinecone, Redis Enterprise Cloud
  - You can control the “chunking” of your data
    - How many tokens are represented by each vector



Using Knowledge Bases

- "Chat with your document"
  - Basically, automatic RAG
- Integrate it into an application directly
- Incorporate it into an agent
  - "Agentic RAG"